

Lab-1-CECS-475 / stock.cs



mattben765 fixed race conditions and declared events as nullable

1337920 · 38 minutes ago [History](#)

98 lines (95 loc) · 3.95 KB

Code

Blame

Raw



```

1    // stock.cs
2    // Author: Peter Lam
3    using System.Collections.Generic;
4    using System.Text;
5    using System.Threading;
6    using System;
7
8    namespace Stock
9    {
10       //-----
11       public class Stock
12       {
13           public event EventHandler<StockNotification>? StockEvent;
14           //Name of our stock.
15           private string _name;
16           //Starting value of the stock.
17           private int _initialValue;
18           //Max change of the stock that is possible.
19           private int _maxChange;
20           //Threshold value where we notify subscribers to the event.
21           private int _threshold;
22           //Amount of changes the stock goes through.
23           private int _numChanges;
24           //Current value of the stock.
25           private int _currentValue;
26           private readonly Thread _thread;
27           public string StockName { get => _name; set => _name = value; }
28           public int InitialValue { get => _initialValue; set => _initialValue = value; }
29           public int CurrentValue { get => _currentValue; set => _currentValue = value; }

```

```

30     public int MaxChange { get => _maxChange; set => _maxChange = value; }
31     public int Threshold { get => _threshold; set => _threshold = value; }
32     public int NumChanges { get => _numChanges; set => _numChanges = value; }
33     //-----
34     /// <summary>
35     /// Stock class that contains all the information and changes of the stock
36     /// </summary>
37     /// <param name="name">Stock name</param>
38     /// <param name="startingValue">Starting stock value</param>
39     /// <param name="maxChange">The max value change of the stock</param>
40     /// <param name="threshold">The range for the stock</param>
41     public Stock(string name, int startingValue, int maxChange, int threshold)
42     {
43         _name = name;
44         _initialValue = startingValue;
45         _currentValue = InitialValue;
46         _maxChange = maxChange;
47         _threshold = threshold;
48         this._thread = new Thread(new ThreadStart(Activate));
49         _thread.Start();
50     }
51     //-----
52     /// <summary>
53     /// Activates the threads synchronizations
54     /// </summary>
55     public void Activate()
56     {
57         for (int i = 0; i < 25; i++)
58         {
59             Thread.Sleep(500); // 1/2 second
60             ChangeStockValue();
61         }
62     }
63     //-----
64     // delegate
65     // public delegate void StockNotification(String stockName, int currentValue)
66     // event
67     // public event StockNotification ProcessComplete;
68     //-----
69     /// <summary>
70     /// Changes the stock value and also raising the event of stock value change
71     /// </summary>
72     public void ChangeStockValue()
73     {
74         var rand = new Random();
75         CurrentValue += rand.Next(1, MaxChange);
76         NumChanges++;

```

```
77         if ((CurrentValue - InitialValue) > Threshold)
78             { //RAISE THE EVENT
79                 StockEvent?.Invoke(this, new StockNotification(StockName, CurrentVal
80             })
81         }
82         //-----
83     }
84
85     public class StockNotification : EventArgs
86     {
87         public string StockName { get; set; }
88         public int CurrentValue { get; set; }
89         public int NumChanges { get; set; }
90
91         public StockNotification(string stockName, int currentValue, int numChanges)
92         {
93             this.StockName = stockName;
94             this.CurrentValue = currentValue;
95             this.NumChanges = numChanges;
96         }
97     }
98 }
```

Lab-1-CECS-475 / stockbroker.cs



mattben765 updated writing logic

7455522 · 6 minutes ago [History](#)

90 lines (83 loc) · 3.53 KB

Code

Blame

Raw



```

1    // stockbroker.cs implementation
2    // Author: Matthew Bennett
3
4    using System;
5    using System.Collections.Generic;
6    using System.IO;
7    using System.Threading;
8    using System.Threading.Tasks; // added necessary statements
9
10   namespace Stock { // proper namespace
11       public class StockBroker
12       {
13           public string BrokerName { get; set; }
14           // The broker holds a list of Stocks (though not strictly needed in this lab
15           public List<Stock> stocks = new List<Stock>();
16           // We'll write to "Lab1_output.txt" in the same folder as the .exe
17           readonly string destPath =
18               System.IO.Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "Lab1_Output.t
19           // We'll print this header in both console & file
20           public string titles =
21               "Broker".PadRight(10) +
22               "Stock".PadRight(15) +
23               "Value".PadRight(10) +
24               "Changes".PadRight(10) +
25               "Date and Time";
26           private static bool _headerWritten = false; // flag to help with race condit
27           private static readonly SemaphoreSlim _fileLock = new SemaphoreSlim(1,1);
28
29           public StockBroker(string brokerName)
30           {
31               BrokerName = brokerName;

```

```

31         _brokerName = _brokerName;
32         if (!_headerWritten)
33         {
34             // Print the header to console
35             Console.WriteLine(titles);
36             // Overwrite (false) the file with this same header once
37             using (FileStream fs = new FileStream(destPath, FileMode.Create, Fil
38             using (StreamWriter outputFile = new StreamWriter(fs))
39             {
40                 outputFile.WriteLine(titles);
41             }
42             _headerWritten = true; ; // prevents brokers from rewriting
43         }
44     }
45
46     public void AddStock(Stock stock)
47     {
48         stocks.Add(stock);
49         // Subscribe to the stock's event using our event handler
50         stock.StockEvent += EventHandler; // subscription
51     }
52
53     private async void EventHandler(object? sender, EventArgs e) // async void f
54     { if (sender is not null)
55         // The second parameter needs to be cast to StockNotification
56         await Helper(sender, (StockNotification) e);
57     }
58
59     public async Task Helper(object sender, StockNotification e) // method needs
60     {
61         // We could cast the sender back to Stock if we needed more info
62         Stock newStock = (Stock)sender;
63         // Construct the output line
64         string message =
65             $"{BrokerName.PadRight(10)}" +
66             $"{e.StockName.PadRight(15)}" +
67             $"{e.CurrentValue.ToString().PadRight(10)}" +
68             $"{e.NumChanges.ToString().PadRight(10)}" +
69             $"{DateTime.Now}";
70         await _fileLock.WaitAsync(); //force threads to wait!!!
71         try
72         {
73             // Append this line to the output file
74             using (FileStream fs = new FileStream(destPath, FileMode.Append, Fil
75             using (StreamWriter outputFile = new StreamWriter(fs))
76             {
77                 await outputFile.WriteLineAsync(message);
78             }
79         }
80     }

```

```
78         r
79
80         Console.WriteLine(message);
81     }
82     catch (IOException)
83     {
84         // Handle or log any file I/O exceptions if needed
85         Console.WriteLine($"ERROR WRITING");
86     }
87     finally { _fileLock.Release(); } // release lock
88 }
89 }
90 }
```